PL/SQL Training

SESSION 4: Creating Stored Procedures , Functions and Packages

DAVA.X ACADEMY SESSIONS

Svetlana Sura & Iulian Aurel Cozma

10–20 JUNE 2025

SESSION 1

Creating Stored Procedures , Functions and Packages

- Differentiate between anonymous blocks and subprograms
- Create a simple procedure and invoke it from an anonymous block
- Create a simple function
- Create a simple function that accepts a parameter Table Functions
- Differentiate between procedures and functions
- Introduction into Package
- Package definition
- Package body
- Package-level data
- Subprogram overloading

Procedure

A procedure is a named PL/SQL block that performs a specific task. It can accept input and output parameters, execute SQL and PL/SQL statements, and optionally return results via OUT parameters (but not via RETURN, which is used in functions).

Basic Syntax of a Procedure:

```
CREATE [OR REPLACE] PROCEDURE procedure_name (
            [parameter1 IN | OUT | IN OUT datatype,
            [parameter2 IN | OUT | IN OUT datatype, ...]
                        )
[AUTHID DEFINER | CURRENT_USER ]
[ACCESSIBLE BY (program_unit_list)]
IS
  -- Variable declarations
BEGIN
  -- Executable statements
EXCEPTION
  -- Exception handling (optional)
END procedure_name;
```

Parameter Modes	
IN	Default mode. Value passed into the procedure.
OUT	Returns a value back to the caller.
IN OUT	Both input and output. Passed in, modified, and returned.

Where:	
AUTHID	Determines whether the procedure will execute with the privileges of the definer (owner) of the procedure or of the current user.
ACCESSIBLE BY	ACCESSIBLE BY clause (Oracle Database 12c) Restricts access to this function to the program units listed inside the parentheses.

CALL a procedure in Oracle

To CALL a procedure in Oracle, you can use a few different methods depending on the context: SQL*Plus, Oracle SQL Developer, or within PL/SQL code.

#	Methods	Syntax/Sample
1	Using EXEC or EXECUTE in SQL Developer / SQL*Plus	EXEC procedure_name(parameter1, parameter2);

2	Using an Anonymous PL/SQL Block - with OUT parameter	DECLARE
		  v_bonus NUMBER;
		BEGIN
		  calculate_bonus(101, v_bonus);
		  DBMS_OUTPUT.PUT_LINE('Bonus: ' || v_bonus);
		END;

3	Using CALL Statement (ANSI style)	CALL procedure_name(parameters);

4	Calling from Another Procedure	CREATE OR REPLACE PROCEDURE process_bonus IS
		  v_bonus NUMBER;
		BEGIN
		  calculate_bonus(101, v_bonus);
		  -- Further processing
		END;

Function

A *function* is a module that returns data through its RETURN clause.
Structure of a Function is in the bellow image where:

DETERMINISTIC clause
These are functions that always return the same result for the same input. Must not include queries or changing state

PARALLEL_ENABLE clause
Is an optimization hint that enables the function to be executed in parallel when called from within a SELECT statement.

PIPELINED clause
Specifies that the results of this table function should be returned iteratively via the PIPE ROW command.

RESULT_CACHE clause
Specifies that the input values and result of this function should be stored in the new function result cache. This feature, introduced in Oracle Database 11g.

AGGREGATE clause
Is used when you are defining your own aggregate function.

EXTERNAL clause
Is used to define the function as implemented "externally"—i.e., as C code.

Structure of a Function

```
FUNCTION [schema.]name[( parameter[, parameter...] ) ]
RETURN return_datatype

[AUTHID DEFINER | CURRENT_USER]
[DETERMINISTIC]
[PARALLEL_ENABLE ...]
[PIPELINED]
[RESULT_CACHE ...]
[ACCESSIBLE BY (program_unit_list)
[AGGREGATE ...]
[EXTERNAL ...]

IS
  [declaration statements]
BEGIN
   executable statements
[EXCEPTION
    exception handler statements]
END [name];
```

Restrictions for Functions in SQL

Functions used inside SQL must not:

- Perform DML (INSERT/UPDATE/DELETE)
- Use DBMS_OUTPUT
- Contain COMMIT or ROLLBACK

Functions vs Procedures

Feature	Function	Procedure
Returns a value	✅ Yes (via RETURN)	❌ No (can use OUT/IN OUT params)
Can be used in SQL	✅ Yes (with restrictions)	❌ No
Intended usage	Calculation, return single result	Perform actions, complex logic
Call style	Can be used in **SELECT**, :=, etc.	Must be called in PL/SQL block
RETURN keyword required	✅ Yes	❌ No

When to Use What?

Goal	Use
Return a single computed value	✅ Function
Execute multiple operations / complex flow	✅ Procedure
Want to call from SQL directly	✅ Function
Need to modify data (DML inside)	✅ Procedure
Return multiple values	✅ Procedure (with OUT params)

Packaged

A *package* is a grouping or packaging together of elements of PL/SQL code into a named scope.

It is a collection of related procedures, functions, variables, constants, cursors, and types stored together as a single unit in the Oracle database.

A package consists of two parts:

Package specification – the public interface

The package specification contains the definition or specification of all the publicly available elements in the package that may be referenced outside of the package. The specification is like one big declaration section; it does not contain any PL/SQL blocks or executable code. If a specification is well designed, a developer can learn from it everything necessary to use the package. There should never be any need to go "behind" the interface of the specification and look at the implementation, which is in the body.

Package body– the implementation

The body of the package contains all the code required to implement elements defined in the package specification. The body may also contain private elements that do not appear in the specification and therefore cannot be referenced outside of the package.

Package Specification	Package Body

CREATE OR REPLACE PACKAGE PackageName
IS
  g_variable NUMBER := 0.10; -- global variable
 	
  PROCEDURE Procedure1(p_id NUMBER);
  FUNCTION Function1(p_id NUMBER)
  RETURN VARCHAR2;
END employee_pkg;
	CREATE OR REPLACE PACKAGE BODY PackageName
	IS
	  PROCEDURE Procedure1(p_id NUMBER)
	  IS
	  BEGIN
	  .....
	  END;
	
	  FUNCTION Function1 (p_id NUMBER)
	  RETURN VARCHAR2
	  IS
	  v_name VARCHAR2(100);
	  BEGIN
	  ...
	  RETURN v_name;
	  END;
	
	END PackageName;

Overloading in PL/SQL

Overloading allows you to define multiple procedures or functions with the same name, but with different parameter lists (in number, order, or data types).

This feature is only allowed inside PL/SQL packages.

📦 Where Can You Overload?

Structure	Supports Overloading?
Standalone procedure/function	❌ No
Package procedure/function	✅ Yes
Object type methods	✅ Yes

Good to know

🔒 Public vs Private

Elements declared in the spec are public and accessible from outside the package.

Elements declared only in the body are private and used only internally.

🧠 Best Practices

Use packages to group logically related logic

Keep spec minimal and stable (changes to body don't invalidate dependents)

Use global variables carefully (they are session-based)

Document each element in the spec

Summary & Homework

THEORY

PL/SQL 101 Part 11

Efficient Function Calls From SQL (Part 6) : Function-Based Indexes

PRACTICE

Get Started with Table Functions 1

HOMEWORK

Quiz on CREATE PROCEDURE Statement

Quiz on Packages

Quiz on Package-Level Data

Quiz on Understanding the PLSQL Package Body

TAKEAWAY

Pipelined Table Functions

Quiz on Table Functions

Quiz on Table Functions